Pontificia Universidad Católica Madre y Maestra

Facultad de Ciencias de la Ingeniería

Departamento de Tecnologías de la Información y Comunicación

Sistema de Gestión y Control de Eventos Académicos

Segundo Parcial

ICC-352 – Programación Web

Presentado por:

Randy Alexander Germosén Ureña
Hugo Fernando Concepción López

Profesor:

Carlos Alfredo Camacho Guerrero

Marzo 2026

Repositorio (GitHub):

Link al repositorio

Video de la demo del proyecto:

Link al video

Índice

1. Introducción **2**

2. Objetivos **2**
2.1. Objetivo General 2
2.2. Objetivos Específicos 2

3. Herramientas y Tecnologías Utilizadas **2**
3.1. Backend 2
3.2. Frontend 3
3.3. Infraestructura y Despliegue 3

4. Arquitectura del Sistema **4**

5. Modelo de Datos **6**
5.1. Usuario 6
5.2. Evento 6
5.3. Inscripción 7

6. Funcionalidades Implementadas **7**
6.1. Autenticación y Registro 7
6.2. Gestión de Eventos 8
6.3. Inscripción y Códigos QR 9
6.4. Escaneo QR y Control de Asistencia 10
6.5. Administración de Usuarios 11
6.6. Dashboard y Estadísticas 12
6.7. Navegación por Roles 13

7. Despliegue con Docker **13**

8. Conclusiones **13**

1. Introducción

El presente documento describe el desarrollo del **Sistema de Gestión y Control de Eventos Académicos**, un proyecto realizado como parte de la evaluación del segundo parcial de la asignatura ICC-352 Programación Web. El sistema permite la creación, organización y seguimiento de eventos académicos dentro de un entorno universitario, proporcionando herramientas para administradores, organizadores y participantes.

La aplicación aborda la necesidad de digitalizar y centralizar la gestión de eventos académicos, ofreciendo funcionalidades como registro de asistencia mediante códigos QR, estadísticas en tiempo real y un panel de control diferenciado por roles de usuario.

2. Objetivos

2.1. Objetivo General

Desarrollar una aplicación web completa para la gestión de eventos académicos que permita la administración de usuarios, creación de eventos, inscripción de participantes y control de asistencia.

2.2. Objetivos Específicos

- Implementar un sistema de autenticación y autorización basado en roles (Administrador, Organizador, Participante).
- Desarrollar un CRUD completo para la gestión de eventos con estados de publicación y cancelación.
- Integrar un mecanismo de inscripción con generación de códigos QR únicos por participante.
- Implementar escaneo de códigos QR para el registro automático de asistencia.
- Crear dashboards con estadísticas visuales diferenciadas por rol de usuario.
- Desplegar la aplicación mediante contenedores Docker con compilación multi-stage.

3. Herramientas y Tecnologías Utilizadas

3.1. Backend

- **Java 21**: Lenguaje de programación principal del proyecto.
- **Javalin 7.0.1**: Framework web ligero para Java y Kotlin, utilizado para el enrutamiento y manejo de peticiones HTTP.

- **Hibernate ORM 6.4.4**: Framework de mapeo objeto-relacional para la persistencia de datos.
- **H2 Database**: Base de datos embebida en modo servidor (AUTO_SERVER=TRUE) que permite acceso concurrente.
- **Thymeleaf 3.1.2**: Motor de plantillas del lado del servidor para la generación de vistas HTML.
- **BCrypt**: Algoritmo de hashing utilizado para el almacenamiento seguro de contraseñas.
- **Gson**: Librería de Google para serialización y deserialización JSON, utilizada para pasar datos a gráficos.
- **Jackson**: Procesador JSON requerido por Javalin para las respuestas `ctx.json()`.

3.2. Frontend

- **Bootstrap 5.3.3**: Framework CSS para diseño responsivo y componentes de interfaz.
- **Font Awesome 6.5.1**: Biblioteca de iconos vectoriales.
- **Chart.js 4.4.1**: Librería JavaScript para la visualización de datos con gráficos interactivos.
- **html5-qrcode**: Librería para escaneo de códigos QR mediante la cámara del dispositivo.
- **qrcodejs**: Generador de códigos QR del lado del cliente.
- **Fetch API**: Interfaz nativa de JavaScript para comunicación asíncrona con el servidor.

3.3. Infraestructura y Despliegue

- **Gradle 9.2.1**: Sistema de automatización de compilación.
- **ShadowJar**: Plugin de Gradle para crear un JAR único ejecutable con todas las dependencias.
- **Docker**: Plataforma de contenedores para el empaquetado y despliegue de la aplicación.
- **Docker Compose**: Herramienta para la definición y ejecución de aplicaciones multi-contenedor.
- **Nginx Proxy Manager**: Proxy inverso para gestión de dominios y certificados TLS.

4. Arquitectura del Sistema

El sistema sigue una arquitectura MVC (Modelo-Vista-Controlador) organizada en las siguientes capas:

- **Modelos**: Entidades JPA que representan las tablas de la base de datos (Usuario, Evento, Inscripción) con sus enumeraciones asociadas (RolUsuario, EstadoInscripción).
- **Servicios**: Capa de lógica de negocio que encapsula las operaciones de persistencia y validaciones (UsuarioService, EventoService, InscripcionService).
- **Controladores**: Manejan las peticiones HTTP y coordinan entre servicios y vistas (AuthController, EventoController, InscripcionController, AdminController, DashboardController).
- **Filtros**: Middleware de autorización que protege las rutas según el rol del usuario autenticado (AuthFilter).
- **Vistas**: Plantillas Thymeleaf que renderizan la interfaz de usuario con un layout común basado en sidebar.

```
app/src/main/
├── java
│   └── pucmm
│       └── application
│           ├── controllers
│           ├── filters
│           ├── models
│           │   └── enums
│           ├── services
│           └── utils
└── resources
    ├── META-INF
    ├── public
    │   ├── css
    │   └── js
    └── templates
```

Figura 1: Estructura de directorios del proyecto

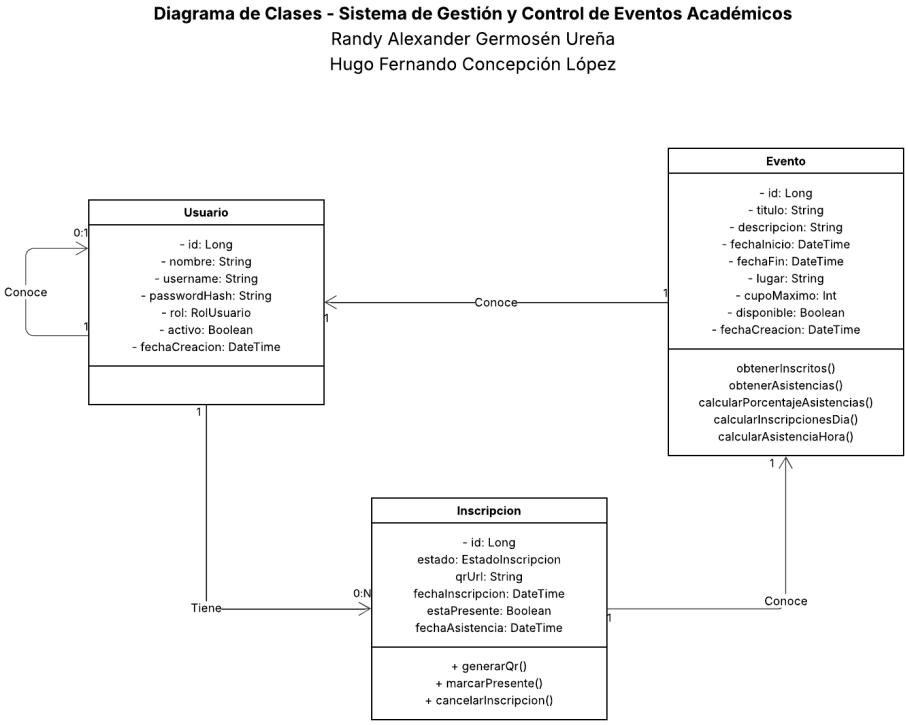

Figura 2: Diagrama de clases del sistema

5. Modelo de Datos

El sistema cuenta con tres entidades principales mapeadas mediante Hibernate ORM:

5.1. Usuario

Representa a los usuarios del sistema. Contiene campos para nombre, nombre de usuario (*username*), hash de contraseña, rol, estado activo y una referencia opcional al administrador que lo creó. El administrador principal del sistema se marca como no eliminable para proteger su integridad.

5.2. Evento

Almacena la información de cada evento académico: título, descripción, fecha, ubicación, capacidad máxima, estado de publicación y cancelación. Mantiene una relación con el organizador que lo creó y con las inscripciones asociadas. Incluye métodos de negocio para calcular estadísticas de asistencia.

5.3. Inscripción

Vincula un participante con un evento. Cada inscripción genera un token QR único (UUID) y registra el estado (activa o cancelada), presencia y hora de asistencia. Una restricción única impide inscripciones duplicadas; si un participante cancela y se reinscribe, se reactiva el registro existente.

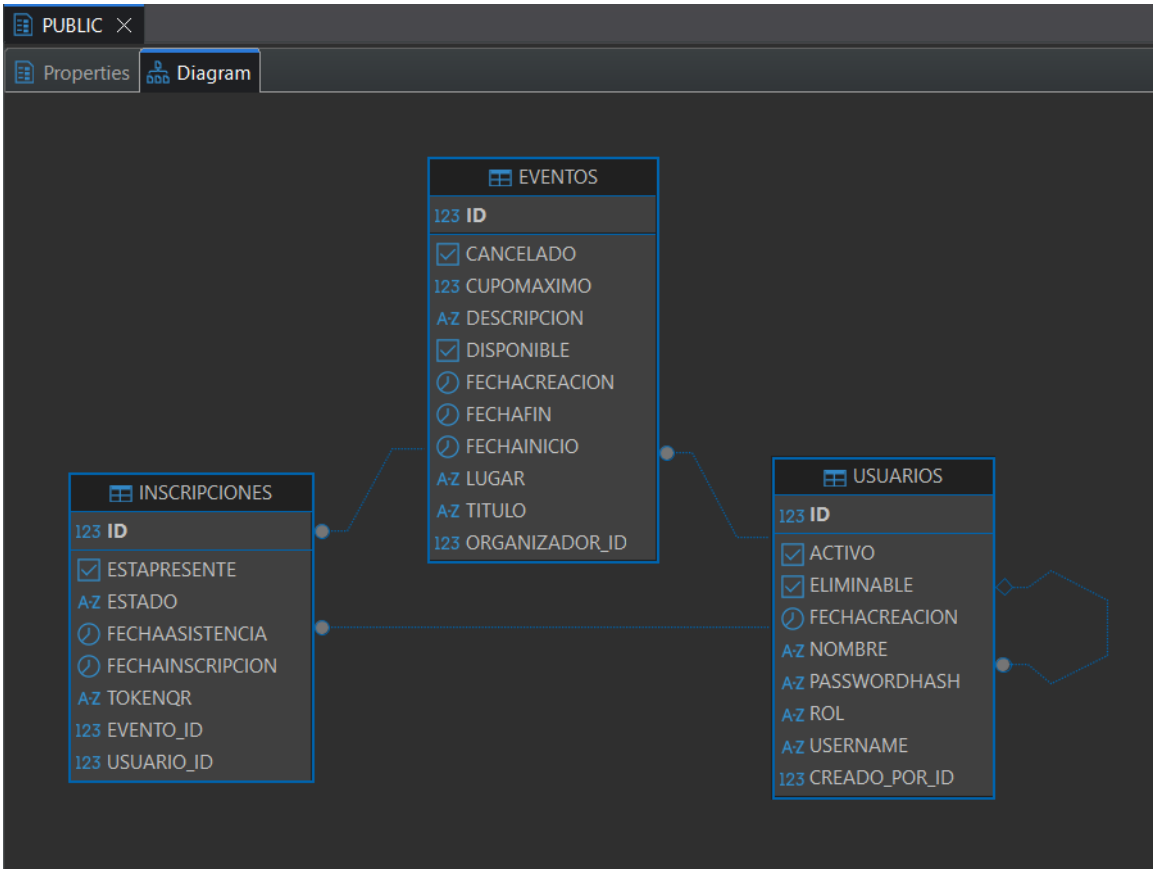

Figura 3: Diagrama de entidad-relación generado desde la base de datos

6. Funcionalidades Implementadas

6.1. Autenticación y Registro

El sistema implementa un mecanismo de autenticación basado en sesiones HTTP. Los usuarios pueden registrarse seleccionando el rol de Organizador o Participante. Las contraseñas se almacenan con hash BCrypt. El administrador principal se crea automáticamente al iniciar la aplicación y no puede ser eliminado ni bloqueado.

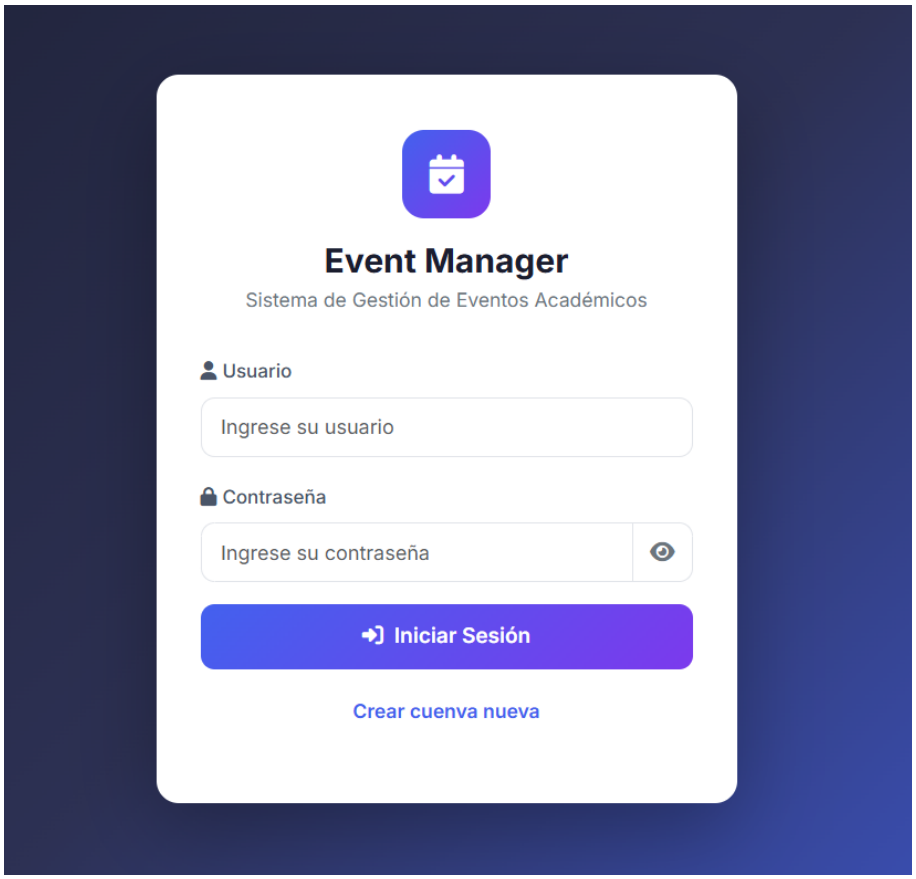

Figura 4: Formulario de inicio de sesión

6.2. Gestión de Eventos

Los organizadores pueden crear, editar, publicar y cancelar eventos. Cada evento tiene capacidad máxima, y solo los eventos publicados son visibles para los participantes. El sistema valida que la fecha del evento sea futura al momento de creación y controla la disponibilidad de cupos.

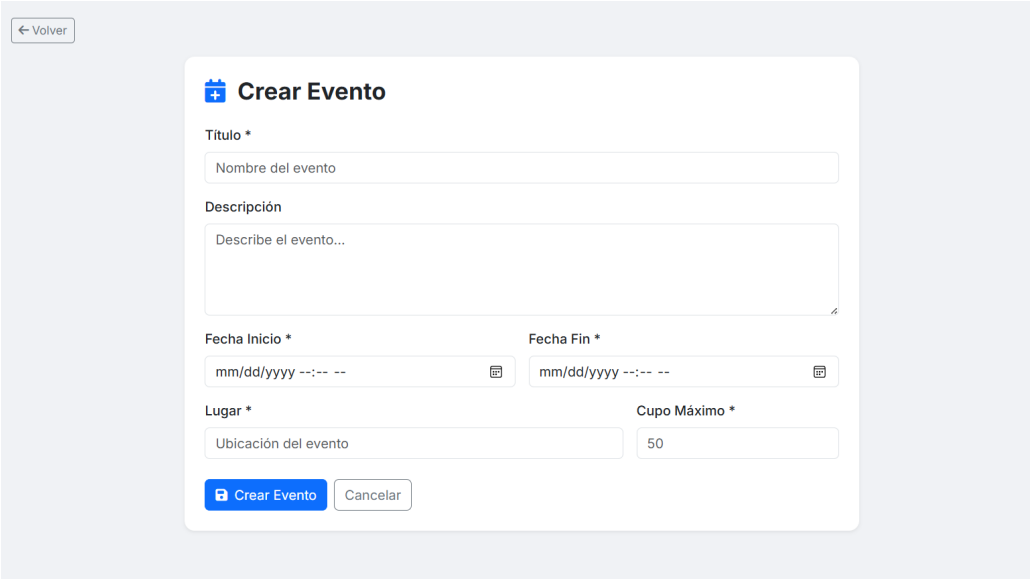

Figura 5: Formulario de creación de eventos

6.3. Inscripción y Códigos QR

Los participantes pueden inscribirse a eventos disponibles. Al inscribirse, se genera automáticamente un código QR único vinculado a un token UUID. Este código sirve como entrada digital al evento. Si un participante cancela su inscripción y decide reinscribirse, el sistema reactiva la inscripción original en lugar de crear un registro duplicado.

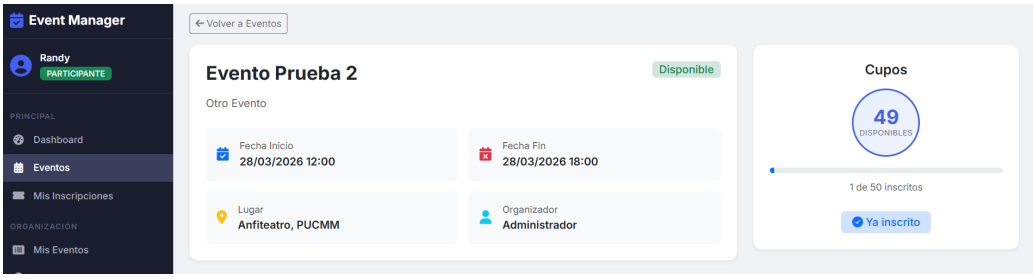

Figura 6: Detalle de evento con opción de inscripción

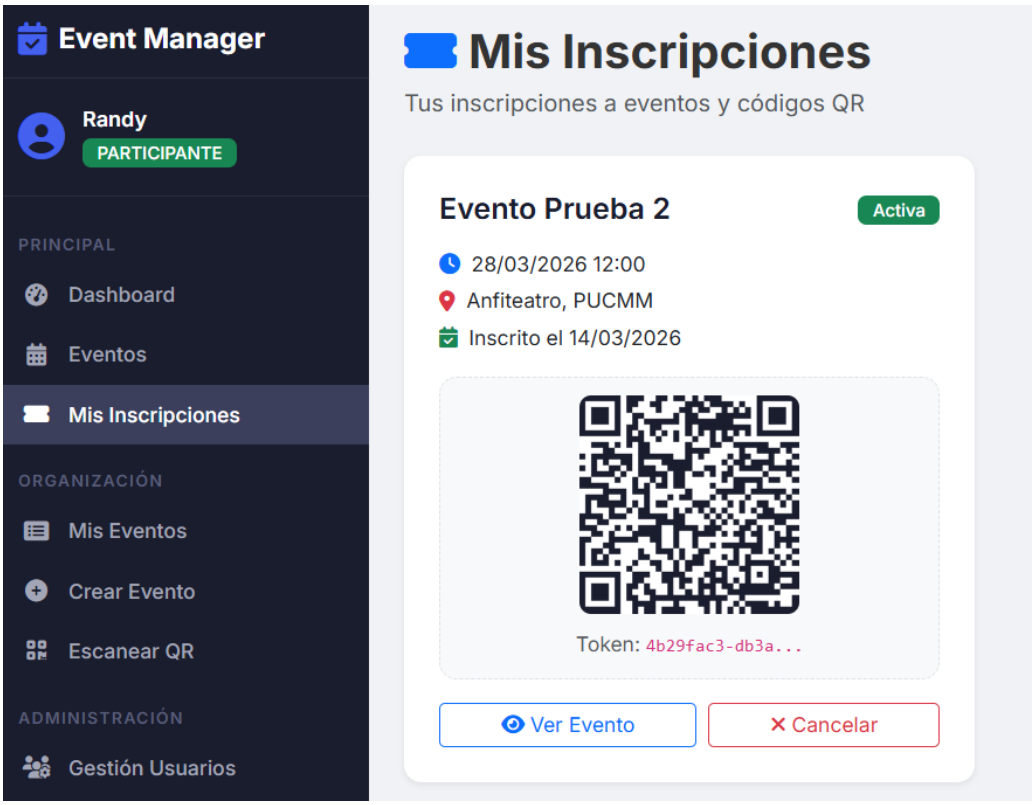

Figura 7: Inscripciones del participante con códigos QR

6.4. Escaneo QR y Control de Asistencia

Los organizadores cuentan con un escáner de códigos QR integrado que utiliza la cámara del dispositivo. Al escanear el código de un participante, el sistema verifica la validez de la inscripción, confirma que el organizador tiene permiso sobre el evento y registra la asistencia con marca de tiempo. También se ofrece la opción de ingresar el token manualmente.

Figura 8: Interfaz de escaneo de códigos QR

Figura 9: Lista de asistencia de un evento

6.5. Administración de Usuarios

El administrador puede crear usuarios directamente asignándoles un rol específico, editar todos los campos de cualquier usuario (incluyendo cambio de contraseña sin mostrar la actual), bloquear usuarios (eliminación lógica) y eliminar usuarios permanentemente de la base de datos. El administrador principal puede ser editado en nombre y contraseña, pero no se puede eliminar, bloquear ni cambiar su rol.

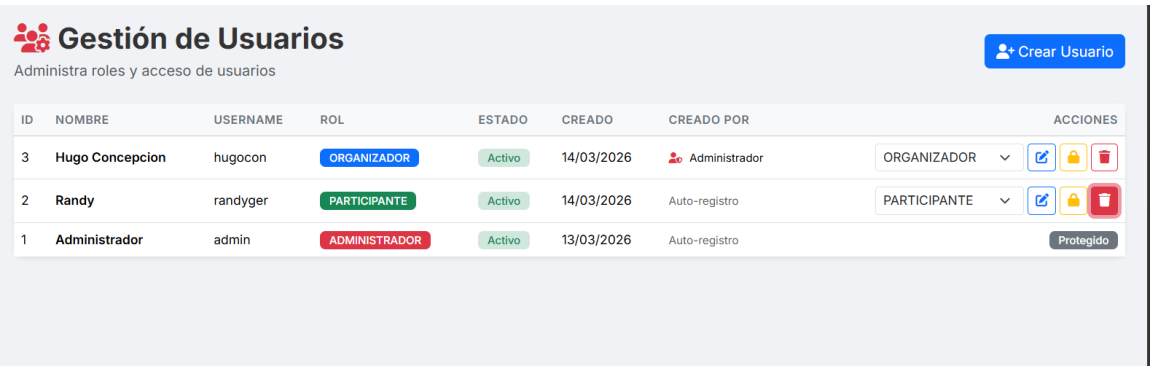

Figura 10: Panel de gestión de usuarios del administrador

6.6. Dashboard y Estadísticas

Cada rol de usuario cuenta con un dashboard personalizado con gráficos interactivos implementados con Chart.js:

- **Administrador**: Distribución de usuarios por rol, estado de eventos, top de eventos por inscripciones y porcentaje global de asistencia.
- **Organizador**: Inscripciones por evento propio y porcentaje de asistencia de sus eventos.
- **Participante**: Estado de sus inscripciones (activas, asistidas, canceladas).

Los datos se serializan en formato JSON desde el controlador y se inyectan en la plantilla mediante atributos `data-*` en elementos HTML ocultos, los cuales son leídos por JavaScript para renderizar los gráficos.

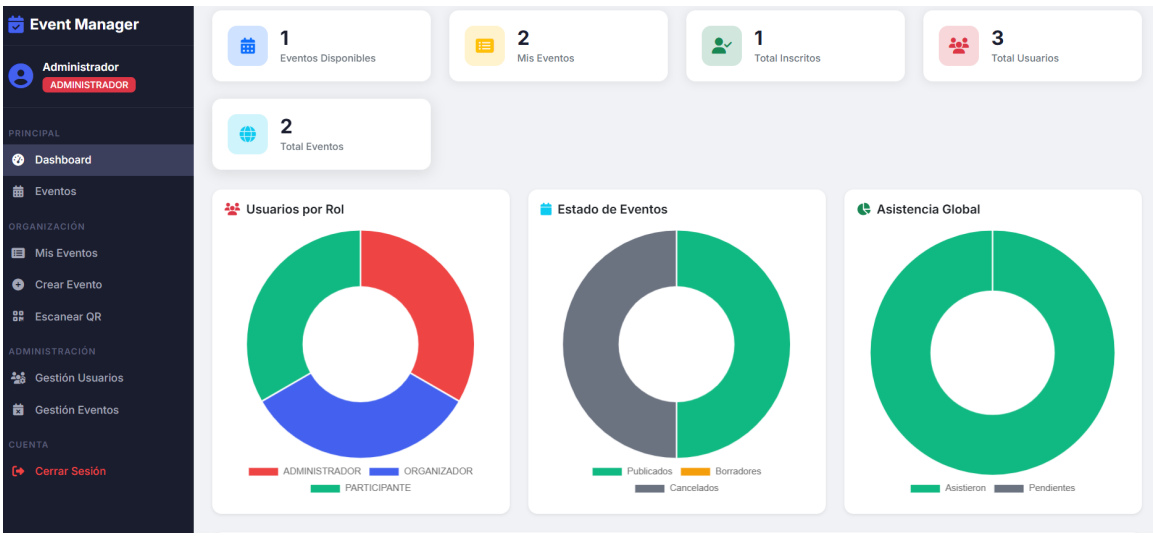

Figura 11: Dashboard con gráficos estadísticos

6.7. Navegación por Roles

La barra lateral de navegación se adapta dinámicamente según el rol del usuario autenticado. Los participantes ven únicamente las secciones de eventos y sus inscripciones; los organizadores acceden además a herramientas de creación de eventos y escaneo QR; mientras que los administradores tienen acceso completo incluyendo la gestión de usuarios y eventos del sistema.

7. Despliegue con Docker

La aplicación se empaqueta mediante un Dockerfile multi-stage:

1. **Etapa de compilación**: Utiliza una imagen de Gradle con JDK 21 Alpine para compilar el proyecto y generar el JAR ejecutable con ShadowJar.
2. **Etapa de ejecución**: Utiliza una imagen ligera de Eclipse Temurin JRE 21 Alpine que solo contiene el runtime necesario, reduciendo significativamente el tamaño de la imagen final.

La base de datos H2 opera en modo servidor con `AUTO_SERVER=TRUE`, lo que permite conexiones concurrentes. Los datos se persisten mediante un volumen Docker nombrado, garantizando que la información sobreviva a reinicios del contenedor.

El contenedor expone el puerto 7000 internamente, mapeado al puerto 30015 del host. Un servidor Nginx Proxy Manager externo se encarga del proxy inverso hacia este puerto y de la generación de certificados TLS para el acceso seguro por HTTPS.

Docker Compose orquesta el despliegue con un solo comando, configurando las variables de entorno necesarias para la base de datos y las credenciales del administrador.

8. Conclusiones

El desarrollo de este sistema permitió aplicar de manera integral los conceptos de programación web estudiados en el curso ICC-352. Se logró construir una aplicación web funcional que abarca desde la persistencia de datos con un ORM hasta la visualización interactiva con gráficos, pasando por la integración de tecnologías de códigos QR para el control de asistencia.

El uso de Javalin como framework web demostró ser una alternativa ligera y eficiente frente a soluciones más complejas, permitiendo un desarrollo ágil sin sacrificar funcionalidad. La combinación de Hibernate ORM con H2 en modo servidor proporcionó una solución de persistencia robusta y fácil de desplegar.

El despliegue con Docker y la compilación multi-stage aseguran que la aplicación sea portable y reproducible en cualquier entorno, facilitando tanto el desarrollo como la puesta en producción.